

ASEN 6044

Homework #2

Bijan Jourabchi

Problem 1:

We are given a prior and measurement likelihood given by

$$P(\theta) = \begin{cases} \alpha e^{-\alpha\theta} & \theta \geq 0 \\ 0 & \theta < 0 \end{cases}$$

$$P(y | \theta) = \frac{\theta^y e^{-\theta}}{y!} \quad y \in \mathbb{Z}^+$$

From Bayes Rule, we know the posterior distribution is given by

$$P(\theta | y) = \frac{P(y | \theta)P(\theta)}{P(y)}$$

Where $P(y)$ is given by the marginalization of the joint distribution $P(\theta, y)$, that is

$$P(y) = \int_{-\infty}^{\infty} P(\theta, y) d\theta = \int_{-\infty}^{\infty} P(y | \theta) P(\theta) d\theta = \int_0^{\infty} \alpha e^{-\alpha\theta} \frac{\theta^y e^{-\theta}}{y!} d\theta$$

Since we are given y , $P(y)$ is a constant and is not dependent on θ . Thus, our posterior distribution will be proportional to the measurement likelihood times the prior.

$$P(\theta | y) \propto \alpha e^{-\alpha\theta} \frac{\theta^y e^{-\theta}}{y!} = \frac{\alpha \theta^y e^{-\theta(1+\alpha)}}{y!}$$

Recall the pdf of a gamma distribution:

$$f(x) = \Gamma(\gamma, \beta) = \frac{\beta^\gamma x^{\gamma-1} e^{-\beta x}}{\Gamma(\gamma)}$$

Where $\Gamma(\gamma)$ is the gamma function. For positive integers, the gamma function returns $\Gamma(n) = (n-1)!$. Since y is a positive integer, it's clear that our posterior distribution is a gamma distribution with parameters $\gamma = y + 1$ and $\beta = \alpha + 1$.

- a) The MMSE estimate for our posterior distribution is the expected value of the distribution.

$$\hat{\theta}_{MMSE} = \mathbb{E}[P(\theta | y)]$$

For a gamma distribution, the expected value is given by $\mathbb{E}[\Gamma(\gamma, \beta)] = \frac{\gamma}{\beta}$. Thus, the MMSE estimate for our posterior distribution is

- b) The MAP estimate for our posterior distribution is the maximum value of our posterior. We can find the maximum by taking the derivative of our posterior with respect to θ and setting equal to 0.

$$\begin{aligned} \frac{d}{d\theta} \left[\frac{\alpha \theta^y e^{-\theta(1+\alpha)}}{y!} \right] &= \frac{\alpha [y \theta^{y-1} e^{-\theta(1+\alpha)} - (1+\alpha) e^{-\theta(1+\alpha)} \theta^y]}{y!} = 0 \\ [y \theta^{y-1} - (\alpha + 1) \theta^y] e^{-\theta(1+\alpha)} &= 0 \rightarrow (\alpha + 1) \theta^y = y \theta^{y-1} \\ \alpha + 1 = y \theta^{-1} &\rightarrow \boxed{\hat{\theta}_{MAP} = \frac{y}{\alpha + 1}} \end{aligned}$$

Problem 2:

We are given that a continuous random variable θ is uniformly distributed on the interval $[0, 1]$, and that measurements are given by $y = \theta + n$ where n 's distribution is given as

$$P(n) = \begin{cases} e^{-n} & n \geq 0 \\ 0 & n < 0 \end{cases}$$

From the following pdf, we can derive our measurement likelihood $P(y | \theta)$. Since our noise is additive, we have $n = y - \theta$. Plugging this into $P(n)$ we have

$$P(y | \theta) = P(y - \theta) = \begin{cases} e^{-y+\theta} & y \geq \theta \\ 0 & y < \theta \end{cases}$$

For a fixed observation y , this is identical to the measurement likelihood, $P(y | \theta)$. Applying Bayes Rule, we can derive an expression for our posterior.

$$P(\theta | y) = \frac{P(y | \theta) P(\theta)}{\int_{-\infty}^{\infty} P(y | \theta) P(\theta) d\theta}$$

The bounds on the posterior must be consistent with the constraints placed on the prior and likelihood. We know $\theta \in [0, 1]$ and $y \geq \theta$. Thus, we know our posterior estimate should be cut off at $\theta = 1$ is $y \geq 1$. However, if $y < 1$, we know that θ cannot be larger than y . Thus, an upper bound on our posterior distribution becomes $u = \min(1, y)$. We can use this upper bound to solve for the normalizing integral in posterior expression.

$$\begin{aligned} \int_{-\infty}^{\infty} P(y | \theta) P(\theta) d\theta &= \int_0^u P(y | \theta) P(\theta) d\theta = \int_0^u e^{-y} e^{\theta} d\theta \\ &= e^{-y} [e^u - 1] \end{aligned}$$

And our posterior becomes

Thus $\hat{\theta}_{MMSE} = \mathbb{E}[P(\theta | y)] = \boxed{\frac{ue^u - e^u + 1}{e^u - 1}}$

b) By taking the derivative of our posterior we can derive an expression for $\hat{\theta}_{MAP}$.

$$\frac{d}{d\theta} \left[\frac{e^\theta}{e^u - 1} \right] = \frac{e^\theta}{e^u - 1}$$

Since this function is monotonic increasing, its maximum must lie at the end of the interval.

Thus, $\boxed{\hat{\theta}_{MAP} = \min(u, 1)}$.

Problem 3:

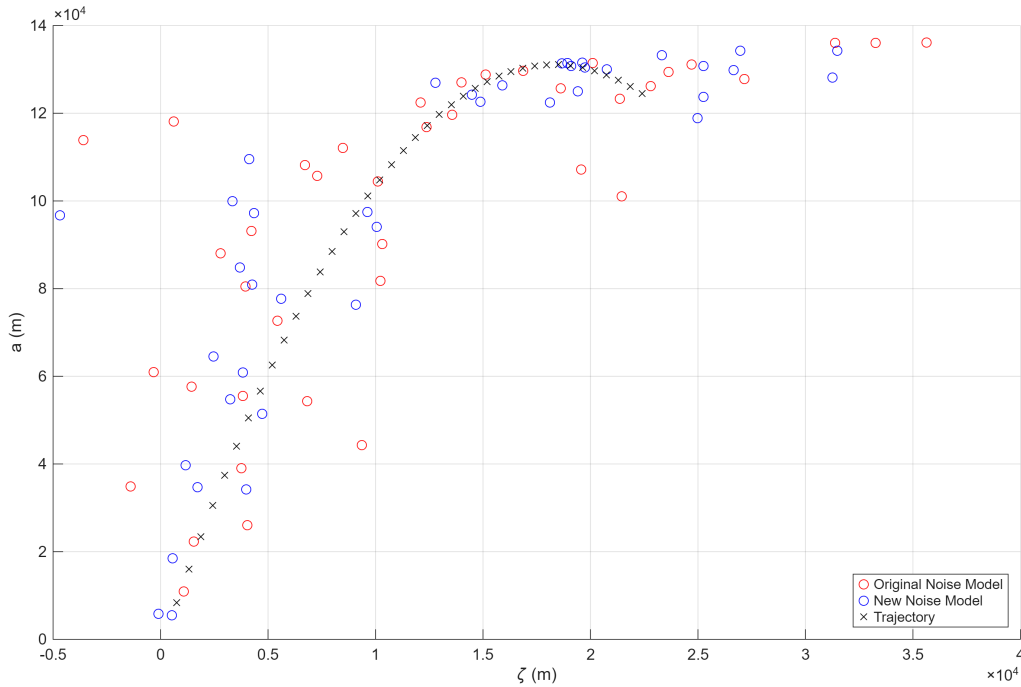


Figure 1: Comparison of Old and New Noisy measurements

- a) From Figure 1 we see generally that the measurements from the new noisy range model draw out a similar shape to that of the old range model. Without any prior knowledge regarding the problem, it even appears that both sets of measurements have the same noise model.
- b) Using the NLS batch estimation code from HW 1 and with an initial $x_0 = [10 \ 30 \ 230 \ 100]^T$ we get the results seen in figure 2. As opposed to HW1, NLS does not work well for the new proposed noise model. This is almost certainly due to the fact that a Student's t-distribution with $\nu = \frac{1}{2}$ does not have a well-defined variance. The non-Gaussian sensor noise present means the least-square cost function is not appropriate for this type of problem. This is due to the new Student's t-distribution having higher order moments that our model is not capturing in R . Thus, the algorithm is not able to make sense of the data it is seeing. This is

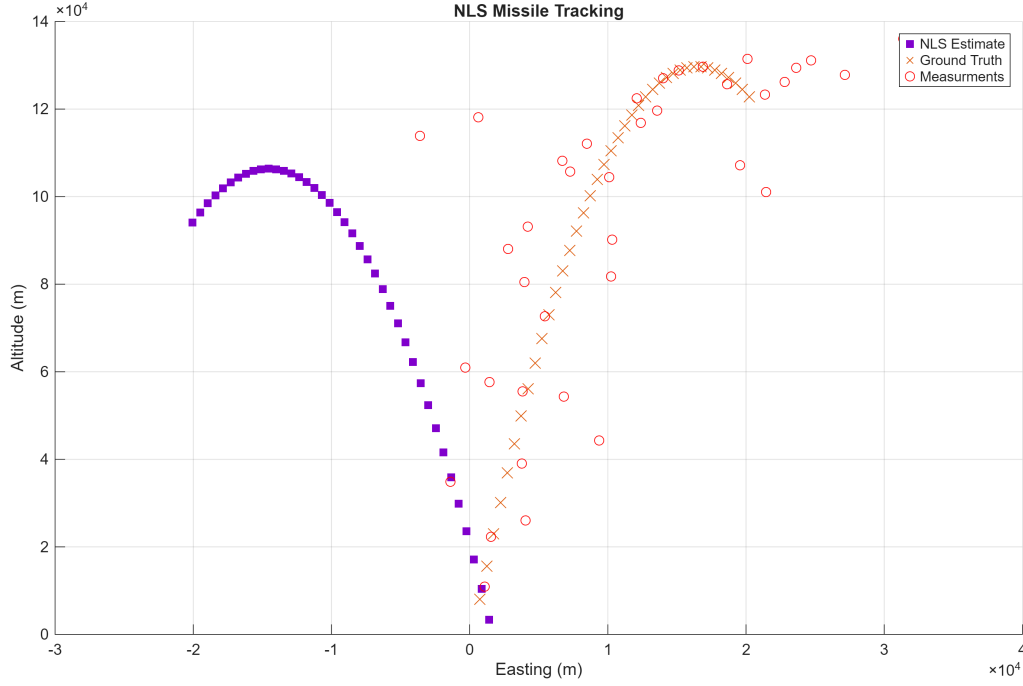


Figure 2: NLS Estimated Trajectory Under New Noise Model

also seen in figure 3 where the range residuals in particular are very large. NLS converged on an estimate of $\hat{x}_{0,NLS} = [1.96 \times 10^3 \ -110.04 \ -3.83 \times 10^3 \ 1.47 \times 10^3]^T$ giving NLS an error of $[-1.72 \times 10^3 \ 210 \ 4 \times 10^3 \ 123.6]^T$. We can also find a resulting covariance, however due to the large residual the covariance estimate is unreliable.

$$P(\hat{x}_{0,NLS}) = \begin{bmatrix} 37.8 & -0.86 & 44.2 & -0.87 \\ -0.86 & 0.051 & -0.314 & 0.045 \\ 44.2 & -0.314 & 98.5 & -0.69 \\ -0.874 & 0.045 & -0.69 & 0.04 \end{bmatrix}$$

- c) The negative log-likelihood function, $\mathcal{L}(x_0; Y_{1:N}) = J_{NLL}$ is defined as the negative logarithm of the measurement likelihood functions, that is

$$J_{NLL} = -\log(P(Y_{1:N} | x_0))$$

To derive an expression for $P(Y_{1:N} | x_0)$, we look at the given measurement model:

$$\rho_k = \sqrt{(L - \xi(t_k))^2 + (a(t_k))^2} + v_{r,k} \quad v_{r,k} \sim \mathcal{T}_{v_{r,k}}\left(0, \nu = \frac{1}{2}\right) \quad (1)$$

$$\theta_k = \arctan\left(\frac{a(t_k)}{L - \xi(t_k)}\right) + v_{b,k} \quad v_{b,k} \sim \mathcal{N}_{v_{b,k}}\left(0, \sigma_b^2 = 0.005\right) \quad (2)$$

Where $v_{r,k}$ and $v_{b,k}$ are the range and bearing noise respectively. To derive the measurement likelihood $P(Y_{1:N} | x_0)$ we assume that the measurements are conditionally independent

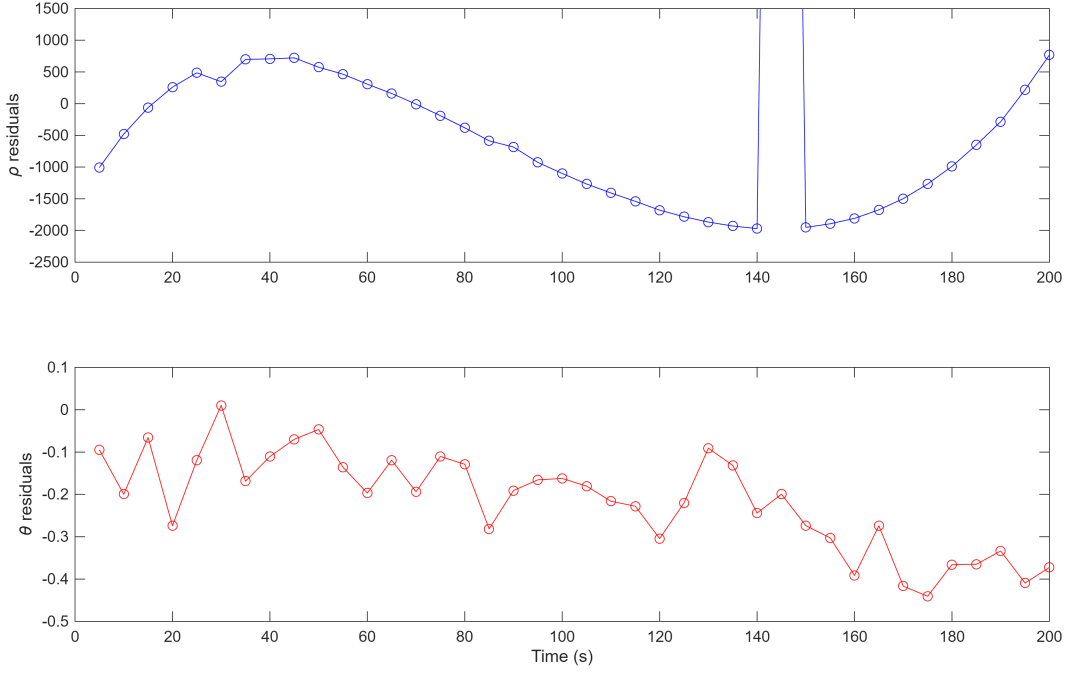


Figure 3: NLS measurement Residuals with ρ Outlier Removed

given x_0 , that is $P(Y_{1:N} | x_0) = \prod_{k=1}^N P(y_k | x_0)$. We also assume that the range and bearing measurements are independent of each other, so our expression further simplifies to the following:

$$P(Y_{1:N} | x_0) = \prod_{k=1}^N P(\rho_k | x_0)P(\theta_k | x_0) \quad (3)$$

Since our noise is additive, solving for the respective probabilities is fairly straightforward. From (1) we can derive $P(\rho_k | x_0)$ as follows:

$$v_{r,k} = \rho_k - \sqrt{(L - \xi(t_k))^2 + (a(t_k))^2}$$

$$P(v_{r,k}) = \frac{\Gamma(\frac{\nu+1}{2})}{\Gamma(\frac{\nu}{2})\sqrt{\nu\pi}} \left(1 + \frac{v_{r,k}^2}{\nu}\right)^{-\frac{\nu+1}{2}}$$

Substituting for $v_{r,k}$ we get the following expression in terms of our initial conditions.

$$P(\rho_k | x_0) = \frac{\Gamma(\frac{\nu+1}{2})}{\Gamma(\frac{\nu}{2})\sqrt{\nu\pi}} \left(1 + \frac{(\rho_k - \sqrt{(L - \xi_0 - \dot{\xi}_0 t_k)^2 + (a_0 + \dot{a}_0 t_k - 0.5 g t_k^2)})^2}{\nu}\right)^{-\frac{\nu+1}{2}} \quad (4)$$

We can follow a similar process to derive $P(\theta_k | x_0)$.

$$\begin{aligned}
 v_{b,k} &= \theta_k - \arctan\left(\frac{a(t_k)}{\xi(t_k)}\right) \\
 P(v_{b,k}) &= \frac{1}{\sigma_b \sqrt{2\pi}} \exp\left\{\frac{-1}{2\sigma_b^2} v_{b,k}^2\right\} \\
 P(\theta_k | x_0) &= \frac{1}{\sigma_b \sqrt{2\pi}} \exp\left\{\frac{-\left[\theta_k - \arctan\left(\frac{a_0 + \dot{a}_0 t_k - 0.5gt_k^2}{L - \xi_0 - \dot{\xi}_0 t_k}\right)\right]^2}{2\sigma_b^2}\right\} \quad (5)
 \end{aligned}$$

Substituting (5) and (4) into (3) and taking the negative logarithm we receive the following expression for our negative log likelihood function:

$$\begin{aligned}
 \mathcal{L}(x_0; Y_{1:N}) &= -\sum_{k=1}^N \log[P(\rho_k | x_0)P(\theta_k | x_0)] \\
 &= -\sum_{k=1}^N \log\left(\frac{\Gamma(\frac{\nu+1}{2})}{\Gamma(\frac{\nu}{2})\sqrt{\nu\pi}} \frac{1}{\sigma_b \sqrt{2\pi}}\right) \\
 &\quad -\sum_{k=1}^N -\frac{\nu+1}{2} \log\left(1 + \frac{(\rho_k - \sqrt{(L - \xi_0 - \dot{\xi}_0 t_k)^2 + (a_0 + \dot{a}_0 t_k - 0.5gt_k^2)})^2}{\nu}\right) \quad (6) \\
 &\quad -\sum_{k=1}^N \frac{-\left[\theta_k - \arctan\left(\frac{a_0 + \dot{a}_0 t_k - 0.5gt_k^2}{L - \xi_0 - \dot{\xi}_0 t_k}\right)\right]^2}{2\sigma_b^2}
 \end{aligned}$$

Both the NLL and NLS cost functions seek to find an x_0 that minimizes the cost function given observations $Y_{1:N}$, however they each approach the problem differently. NLS is defined to minimize the sum of the squared residuals, while maximum likelihood tries to find the x_0 which makes ‘the most sense’ given all the observations. Both assume the measurement noise characteristics are known, NLS uses a least squares cost function, meaning we only consider the 1st/2nd order moments. As we have seen with the Student’s-t distribution, this does not accurately capture higher order moments. Maximum likelihood is robust to non-Gaussian sensor noise unlike the least-squares cost function we have selected for NLS.

d) In order to minimize (6), we take the gradient with respect to x_0 , or $\nabla_{x_0} \mathcal{L}(x_0; Y_{1:N})$, where

$$\nabla_{x_0} = \begin{bmatrix} \frac{\partial}{\partial \xi_0} \\ \frac{\partial}{\partial \dot{\xi}_0} \\ \frac{\partial}{\partial a_0} \\ \frac{\partial}{\partial \dot{a}_0} \end{bmatrix}$$

$$\nabla_{x_0} \mathcal{L}(x_0; Y_{1:N}) = -\nabla_{x_0} \sum_{k=1}^N \log[P(\rho_k | x$$

It's easy to show that the expression simplifies to the following, where the constant terms evaluate to zero.

$$= -\nabla_{x_0} \left[\frac{-\nu + 1}{2} \log \left(1 + \frac{\left(\rho_k - \sqrt{(L - \xi_0 - \dot{\xi}_0 t_k)^2 + (a_0 + \dot{a}_0 - 0.5gt_k^2)^2} \right)^2}{\nu} \right) \right] \quad (7)$$

$$- \nabla_{x_0} \left[\frac{-1}{2} \left[\frac{\theta_k - \arctan \left(\frac{a_0 + \dot{a}_0 t_k - 0.5gt_k^2}{L - \xi_0 - \dot{\xi}_0 t_k} \right)}{\sigma_b} \right]^2 \right]$$

Equation (7) consists of two gradient terms to evaluate. Starting with the first term, we can take each partial derivative independently and stack the result into a vector:

$$\nabla_{x_0} \left[\frac{-\nu + 1}{2} \log \left(1 + \frac{\left(\rho_k - \sqrt{(L - \xi_0 - \dot{\xi}_0 t_k)^2 + (a_0 + \dot{a}_0 - 0.5gt_k^2)^2} \right)^2}{\nu} \right) \right]$$

$$\frac{\partial}{\partial \xi_0} \left[\frac{-\nu + 1}{2} \log \left(1 + \frac{\left(\rho_k - \sqrt{(L - \xi_0 - \dot{\xi}_0 t_k)^2 + (a_0 + \dot{a}_0 - 0.5gt_k^2)^2} \right)^2}{\nu} \right) \right]$$

$$= \frac{-\nu + 1}{2} \frac{1}{\frac{(\rho_k - \sqrt{(L - \xi_0 - \dot{\xi}_0 t_k)^2 + (a_0 + \dot{a}_0 t - 0.5gt_k^2)^2})^2}{\nu} + 1} \frac{2}{\nu} (\rho_k - \sqrt{(L - \xi_0 - \dot{\xi}_0 t_k)^2 + (a_0 + \dot{a}_0 t - 0.5gt_k^2)^2})$$

$$\frac{-1}{-1} [-2(L - \xi_0 - \dot{\xi}_0 t_k)]$$

$$2\sqrt{(L - \xi_0 - \dot{\xi}_0 t_k)^2 + (a_0 + \dot{a}_0 t - 0.5gt_k^2)^2}$$

$$= \frac{2(\rho_k - \sqrt{(L - \xi_0 - \dot{\xi}_0)^2 + (a_0 + \dot{a}_0 t - 0.5gt_k^2)^2})(L - \xi_0 - \dot{\xi}_0 t)}{((\rho_k - \sqrt{(L - \xi_0 - \dot{\xi}_0)^2 + (a_0 + \dot{a}_0 t - 0.5gt_k^2)^2})^2 + \nu)\sqrt{(L - \xi_0 - \dot{\xi}_0)^2 + (a_0 + \dot{a}_0 t - 0.5gt_k^2)^2}}$$

The remaining derivatives follow in a very similar fashion.

$$\frac{\partial}{\partial \dot{\xi}_0} \left[\frac{-\nu + 1}{2} \log \left(1 + \frac{\left(\rho_k - \sqrt{(L - \xi_0 - \dot{\xi}_0 t_k)^2 + (a_0 + \dot{a}_0 - 0.5gt_k^2)^2} \right)^2}{\nu} \right) \right]$$

$$= \frac{2t(\rho_k - \sqrt{(L - \xi_0 - \dot{\xi}_0)^$$

$$\begin{aligned}
& \frac{\partial}{\partial \dot{a}_0} \left[\frac{-\nu + 1}{2} \log \left(1 + \frac{\left(\rho_k - \sqrt{(L - \xi_0 - \dot{\xi}_0 t_k)^2 + (a_0 + \dot{a}_0 - 0.5gt_k^2)^2} \right)^2}{\nu} \right) \right] \\
&= \frac{-2t(\rho_k - \sqrt{(L - \xi_0 - \dot{\xi}_0 t_k)^2 + (a_0 + \dot{a}_0 - 0.5gt_k^2)^2})(a_0 + \dot{a}_0 - 0.5gt_k^2)}{\left(\left(\rho_k - \sqrt{(L - \xi_0 - \dot{\xi}_0 t_k)^2 + (a_0 + \dot{a}_0 - 0.5gt_k^2)^2} \right)^2 + \nu \right) \sqrt{(L - \xi_0 - \dot{\xi}_0 t_k)^2 + (a_0 + \dot{a}_0 - 0.5gt_k^2)^2}}
\end{aligned}$$

Therefore,

$$\begin{aligned}
& \nabla_{x_0} \left[\frac{-\nu + 1}{2} \log \left(1 + \frac{\left(\rho_k - \sqrt{(L - \xi_0 - \dot{\xi}_0 t_k)^2 + (a_0 + \dot{a}_0 - 0.5gt_k^2)^2} \right)^2}{\nu} \right) \right] \\
&= \frac{-\nu + 1}{2} \left[\frac{\frac{2(\rho_k - \sqrt{(L - \xi_0 - \dot{\xi}_0 t_k)^2 + (a_0 + \dot{a}_0 - 0.5gt_k^2)^2})(L - \xi_0 - \dot{\xi}_0 t_k)}{\left(\left(\rho_k - \sqrt{(L - \xi_0 - \dot{\xi}_0 t_k)^2 + (a_0 + \dot{a}_0 - 0.5gt_k^2)^2} \right)^2 + \nu \right) \sqrt{(L - \xi_0 - \dot{\xi}_0 t_k)^2 + (a_0 + \dot{a}_0 - 0.5gt_k^2)^2}}}{\left(\left(\rho_k - \sqrt{(L - \xi_0 - \dot{\xi}_0 t_k)^2 + (a_0 + \dot{a}_0 - 0.5gt_k^2)^2} \right)^2 + \nu \right) \sqrt{(L - \xi_0 - \dot{\xi}_0 t_k)^2 + (a_0 + \dot{a}_0 - 0.5gt_k^2)^2}} \right. \\
&\quad \left. \frac{2t(\rho_k - \sqrt{(L - \xi_0 - \dot{\xi}_0 t_k)^2 + (a_0 + \dot{a}_0 - 0.5gt_k^2)^2})(L - \xi_0 - \dot{\xi}_0 t_k)}{\left(\left(\rho_k - \sqrt{(L - \xi_0 - \dot{\xi}_0 t_k)^2 + (a_0 + \dot{a}_0 - 0.5gt_k^2)^2} \right)^2 + \nu \right) \sqrt{(L - \xi_0 - \dot{\xi}_0 t_k)^2 + (a_0 + \dot{a}_0 - 0.5gt_k^2)^2}} \right. \\
&\quad \left. \frac{-2(\rho_k - \sqrt{(L - \xi_0 - \dot{\xi}_0 t_k)^2 + (a_0 + \dot{a}_0 - 0.5gt_k^2)^2})(a_0 + \dot{a}_0 - 0.5gt_k^2)}{\left(\left(\rho_k - \sqrt{(L - \xi_0 - \dot{\xi}_0 t_k)^2 + (a_0 + \dot{a}_0 - 0.5gt_k^2)^2} \right)^2 + \nu \right) \sqrt{(L - \xi_0 - \dot{\xi}_0 t_k)^2 + (a_0 + \dot{a}_0 - 0.5gt_k^2)^2}} \right. \\
&\quad \left. \frac$$

Therefore,

$$\nabla_{x_0} \left[\frac{-1}{2} \left[\frac{\theta_k - \arctan\left(\frac{a_0 + a_0 t_k - 0.5 g t_k^2}{L - \xi_0 - \xi_0 t_k}\right)}{\sigma_b} \right]^2 \right] = \frac{-1}{2\sigma_b^2} \left[\begin{array}{c} -2[\theta_k - \arctan\left(\frac{a_0 + a_0 t_k - 0.5 g t_k^2}{L - \xi_0 - \xi_0 t_k}\right)](a_0 + a_0 t_k - 0.5 g t_k^2) \\ \frac{([\frac{a_0 + a_0 t_k - 0.5 g t_k^2}{L - \xi_0 - \xi_0 t_k}]^2 + 1)(L - \xi_0 - \xi_0 t_k)^2}{-2t[\theta_k - \arctan\left(\frac{a_0 + a_0 t_k - 0.5 g t_k^2}{L - \xi_0 - \xi_0 t_k}\right)](a_0 + a_0 t_k - 0.5 g t_k^2)} \\ \frac{([\frac{a_0 + a_0 t_k - 0.5 g t_k^2}{L - \xi_0 - \xi_0 t_k}]^2 + 1)(L - \xi_0 - \xi_0 t_k)^2}{-2[\theta_k - \arctan\left(\frac{a_0 + a_0 t_k - 0.5 g t_k^2}{L - \xi_0 - \xi_0 t_k}\right)]} \\ \frac{([\frac{a_0 + a_0 t_k - 0.5 g t_k^2}{L - \xi_0 - \xi_0 t_k}]^2 + 1)(L - \xi_0 - \xi_0 t_k)}{-2t[\theta_k - \arctan\left(\frac{a_0 + a_0 t_k - 0.5 g t_k^2}{L - \xi_0 - \xi_0 t_k}\right)]} \\ \frac{([\frac{a_0 + a_0 t_k - 0.5 g t_k^2}{L - \xi_0 - \xi_0 t_k}]^2 + 1)(L - \xi_0 - \xi_0 t_k)}{-2t[\theta_k - \arctan\left(\frac{a_0 + a_0 t_k - 0.5 g t_k^2}{L - \xi_0 - \xi_0 t_k}\right)]} \end{array} \right] \quad (9)$$

Finally, we can substitute equations (8) and (9) into equation (7) and we are left with an expression for the gradient vector of $\mathcal{L}_{NLL}$

$$\nabla_{x_0} \mathcal{L}(x_0; Y_{1:N}) = \sum_{k=1}^N \left[\frac{\nu - 1}{2} \left[\begin{array}{c} \frac{2(\rho_k - \sqrt{(L - \xi_0 - \xi_0 t)^2 + (a_0 + a_0 t - 0.5 g t_k^2)^2})(L - \xi_0 - \xi_0 t)}{\left(\left(\rho - \sqrt{(L - \xi_0 - \xi_0)^2 + (a_0 + a_0 t - 0.5 g t^2)^2} \right)^2 + \nu \right) \sqrt{(L - \xi_0 - \xi_0)^2 + (a_0 + a_0 t - 0.5 g t^2)^2}} \\ \frac{2t(\rho_k - \sqrt{(L - \xi_0 - \xi_0 t)^2 + (a_0 + a_0 t - 0.5 g t_k^2)^2})(L - \xi_0 - \xi_0 t)}{\left(\left(\rho - \sqrt{(L - \xi_0 - \xi_0)^2 + (a_0 + a_0 t - 0.5 g t^2)^2} \right)^2 + \nu \right) \sqrt{(L - \xi_0 - \xi_0)^2 + (a_0 + a_0 t - 0.5 g t^2)^2}} \\ \frac{-2(\rho_k - \sqrt{(L - \xi_0 - \xi_0 t)^2 + (a_0 + a_0 t - 0.5 g t_k^2)^2})(a_0 + a_0 t - 0.5 g t^2)}{\left(\left(\rho - \sqrt{(L - \xi_0 - \xi_0)^2 + (a_0 + a_0 t - 0.5 g t^2)^2} \right)^2 + \nu \right) \sqrt{(L - \xi_0 - \xi_0)^2 + (a_0 + a_0 t - 0.5 g t^2)^2}} \\ \frac{-2t(\rho_k - \sqrt{(L - \xi_0 - \xi_0 t)^2 + (a_0 + a_0 t - 0.5 g t_k^2)^2})(a_0 + a_0 t - 0.5 g t^2)}{\left(\left(\rho - \sqrt{(L - \xi_0 - \xi_0)^2 + (a_0 + a_0 t - 0.5 g t^2)^2} \right)^2 + \nu \right) \sqrt{(L - \xi_0 - \xi_0)^2 + (a_0 + a_0 t - 0.5 g t^2)^2}} \end{array} \right] \right. \\ \left. + \frac{1}{2\sigma_b^2} \left[\begin{array}{c} -2[\theta_k - \arctan\left(\frac{a_0 + a_0 t_k - 0.5 g t_k^2}{L - \xi_0 - \xi_0 t_k}\right)](a_0 + a_0 t_k - 0.5 g t_k^2) \\ \frac{([\frac{a_0 + a_0 t_k - 0.5 g t_k^2}{L - \xi_0 - \xi_0 t_k}]^2 + 1)(L - \xi_0 - \xi_0 t_k)^2}{-2t[\theta_k - \arctan\left(\frac{a_0 + a_0 t_k - 0.5 g t_k^2}{L - \xi_0 - \xi_0 t_k}\right)](a_0 + a_0 t_k - 0.5 g t_k^2)} \\ \frac{([\frac{a_0 + a_0 t_k - 0.5 g t_k^2}{L - \xi_0 - \xi_0 t_k}]^2 + 1)(L - \xi_0 - \xi_0 t_k)^2}{-2[\theta_k - \arctan\left(\frac{a_0 + a_0 t_k - 0.5 g t_k^2}{L - \xi_0 - \xi_0 t_k}\right)]} \\ \frac{([\frac{a_0 + a_0 t_k - 0.5 g t_k^2}{L - \xi_0 - \xi_0 t_k}]^2 + 1)(L - \xi_0 - \xi_0 t_k)}{-2t[\theta_k - \arctan\left(\frac{a_0 + a_0 t_k - 0.5 g t_k^2}{L - \xi_0 - \xi_0 t_k}\right)]} \\ \frac{([\frac{a_0 + a_0 t_k - 0.5 g t_k^2}{L - \xi_0 - \xi_0 t_k}]^2 + 1)(L - \xi_0 - \xi_0 t_k)}{-2t[\theta_k - \arctan\left(\frac{a_0 + a_0 t_k - 0.5 g t_k^2}{L - \xi_0 - \xi_0 t_k}\right)]} \end{array} \right] \right] \quad (10)$$

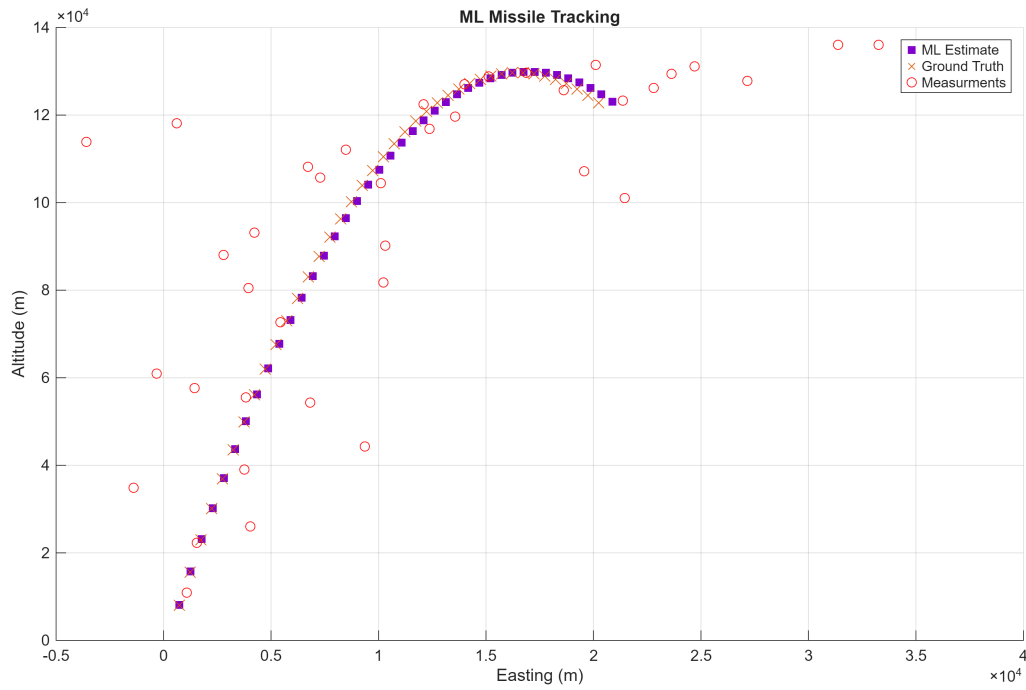


Figure 4: Maximum Likelihood Estimated Trajectory

- e) After some trial and error, I was able to land on an initial $x_0 = [50 \ 300 \ 20 \ 100]^T$ resulting in an estimate of $\hat{x}_{0,ML} = [220.5 \ 103.3 \ 294.3 \ 1.6 \times 10^3]^T$. To do this, I used MATLAB's `fminunc` function, and I specified my own gradient vector specified in equation (10). I found it difficult to land on a good estimate for x_0 without specifying a gradient, and I found MATLAB's `checkGradients` function to be quite helpful in debugging. As expected, maximum likelihood performs much better than NLS as seen in figure 4.

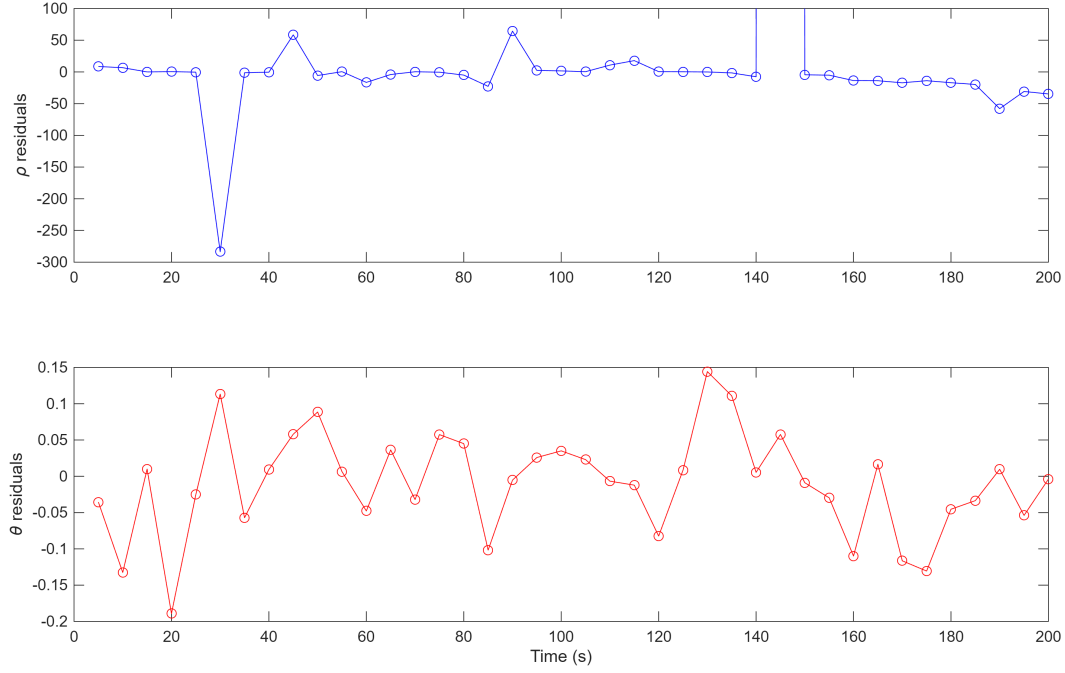


Figure 5: Maximum Likelihood Residuals with ρ Outlier Removed

- f) Comparing the results of part (e) to part (b), we see that the range residuals from ML are significantly smaller than that of NLS as seen in figure 5. The bearing residuals are more or less the same. Since the bearing noise is gaussian, makes sense that NLS would be able to resolve a good bearing estimate from the given data set. In terms of error, the error from the ML estimate was $[13.53 \ -3.33 \ -125.3 \ -0.76]^T$, which is orders of magnitude smaller than that of NLS. If we wanted to go about finding the bias and covariance of $\hat{x}_{0,ML}$, we would have a few options. If it is possible to find an analytic expression for $\hat{x}_{0,ML}$, we could use the following expressions,

$$\text{Bias} = \mathbb{E}[\hat{x}_{0,ML}] - x_0 \quad \text{Cov}(\hat{x}_{0,ML}) = \mathbb{E}[(\tilde{x}_0 - \bar{\tilde{x}}_0)(\dots)^t]$$

Where $\tilde{x}_0 = x_0 - \hat{x}_{0,ML}$ and $\bar{\tilde{x}}_0 = \mathbb{E}[\tilde{x}_0]$. However, due to the dimensionality of the problem and highly non-linear measurement and noise model, finding an analytical expression for $\hat{x}_{0,ML}$ is unlikely. Instead, we could estimate the bias and covariance using Monte Carlo trials. We can fix a value for x_0 and sample N measurements from $P(y \mid x_0)$ for each $m = 1, 2, \dots, M$ MC trial. $\mathbb{E}[\hat{x}_{0,ML}]$ and $\mathbb{E}[\bar{\tilde{x}}_0]$ can then be estimated using the values of $\hat{x}_{0,ML}$ **fminunc** converges to across each MC trial.

3.1 Code for Problem 3a

```

1  %%% Bijan Jourabchi
2  %%% ASEN 6044 HW2 Q3a
3
4  %%% Load Datafile
5  clear; clc; close all
6  load("homework1_missileprob_data.mat")
7  ystackedOG = ystacked;
8  x0trueOG = x0true;
9  load("hw2_missileprob_data.mat")
10
11 N = size(ystackeds,1)/2;
12 np = 2;
13 L = 80000;
14 dt = 5; % sec
15 t = dt:dt:N*dt;
16
17 % Get measurment in x,y coords
18 rho = ystacked(1:2:end);
19 theta = ystacked(2:2:end);
20
21 rhoOG = ystackedOG(1:2:end);
22 thetaOG = ystackedOG(2:2:end);
23
24
25 for i = 1:N
26     aM_1(i) = rho(i)*sin(theta(i));
27     eM_1(i) = L - rho(i)*cos(theta(i));
28
29     aM_2(i) = rhoOG(i)*sin(thetaOG(i));
30     eM_2(i) = L - rhoOG(i)*cos(thetaOG(i));
31     % Get trajectories
32     [eT_1(i),aT_1(i)] = f(x0trueOG,t(i));
33     [eT_2(i),aT_2(i)] = f(x0trueOG,t(i));
34
35 end
36
37 figure(); hold on; grid on
38
39 scatter(eM_1,aM_1,'ro')
40 scatter(eM_2,aM_2,'bo')
41 scatter(eT_2,aT_2,'kx')
42 xlabel('\zeta (m)')
43 ylabel('a (m)')
44
45 legend('Original Noise Model','New Noise
46         Model','Trajectory',Location='southeast')
47 % Create Figures folder if it doesn't exist and save the current figure
48 figDir = fullfile(pwd,'Figures');
49 if ~exist(figDir,'dir')
50     mkdir(figDir);
51 end

```

```

52 % Prepare filename with timestamp to avoid overwriting
53 timestamp = datestr(now,'yyyy-mm-dd_HHMMSS');
54 fname = fullfile(figDir,['missile_measurments' '.png']);
55
56 % Set figure properties for saving
57 fig = gcf;
58 set(fig,'Color','w','PaperPositionMode','auto');
59
60 % Save as PNG (use -r300 for high resolution)
61 print(fig,'-dpng','-r300',fname);
62
63 function [xik,ak] = f(x0,t)
64
65     xi0 = x0(1);
66     xid0 = x0(2);
67     a0 = x0(3);
68     ad0 = x0(4);
69     g = 9.81;
70
71     xik = xi0 + xid0*t;
72     ak = a0 + ad0*t - 0.5*g*t^2;
73 end

```

3.2 Code for Problem 3b

```

1  %%% Bijan Jourabchi
2  %%% ASEN 6044 HW2 Q3
3
4  %%% Load Datafile
5  clear; clc; close all
6  load("hw2_missileprob_data.mat")
7
8  %%% Get Noise Covar matrix
9
10 N = size(ystacked,1)/2;
11 np = 2;
12 L = 80000;
13 dt = 5; % sec
14 t = dt:dt:N*dt;
15
16 R = [70 0; 0 0.005];
17 Rb = kron(eye(N), R);
18
19 %%% Whitening transformation
20
21 S = chol(Rb);
22 ya = S*ystacked;
23
24 %%% NLS
25 x0g = [10;30;230;100];
26 %x0g = [235;-150;200;2000];
27 %x0g = x0true;
28

```

```

29 % NLS
30 itr = 0;
31 maxitr = 100;
32 maxalph = 10;
33 converged = false;
34
35 while itr < maxitr && ~converged
36     % Setup NLS
37     yc = zeros(np,N);
38     H = zeros(np*N,4);
39
40     for k = 1:N
41
42         yc(:,k) = h(x0g,t(k),R, L,true);
43
44         %[xi,a] = f(x0g,t(k));
45         H(2*k-1 : 2*k, :) = jcb(x0g,L,t(k));
46     end
47     yc = S*yc(:);
48     H = S*H;
49
50     % Cost fxn
51
52     residuals = ya - yc;
53     Jcurr = residuals'/Rb*residuals;
54     dx = inv(H'*inv(Rb)*H)*H'*inv(Rb)*(ya - yc);
55     alph = 1;
56     alphasplts = 0;
57     while alphasplts < maxalph
58
59         xn0 = x0g + alph*dx;
60
61
62
63         yc = zeros(np,N);
64         for k = 1:N
65             yc(:,k) = h(xn0,t(k),R, L,true);
66         end
67         yc = S*yc(:);
68
69         Jn = (ya - yc)'/Rb*(ya - yc);
70
71         if abs(Jcurr - Jn) < 50
72             converged = true;
73             break;
74         end
75
76         if Jn > Jcurr
77             alph = alph/2;
78         else
79             x0g = xn0;
80             break;
81         end
82         alphasplts = alphasplts + 1;

```

```

83     end
84     itr = itr + 1;
85 end
86
87 err = x0true - x0g %% ERROR
88
89 for k = 1:N
90
91     H(2*k-1 : 2*k, :) = jcb(x0g,80000,t(k));
92 end
93
94 P = inv(H'*inv(Rb)*H);
95 plot_NLS(x0true,x0g,ystacked,t,Rb,L)
96
97
98 %%% Measurment function
99 function [y] = h(x0,t,R,L,noise)
100
101     if noise
102         vk = mvnrnd([0,0],R)';
103     else
104         vk = [0 0];
105     end
106
107     [xik,ak] = f(x0,t);
108
109     rho = sqrt((L - xik)^2 + ak^2) + vk(1);
110     theta = atan2(ak,(L-xik)) + vk(2);
111
112     y = [rho;theta];
113 end
114
115 %%% Dynamics function
116 function [xik,ak] = f(x0,t)
117
118     xi0 = x0(1);
119     xid0 = x0(2);
120     a0 = x0(3);
121     ad0 = x0(4);
122     g = 9.81;
123
124     xik = xi0 + xid0*t;
125     ak = a0 + ad0*t - 0.5*g*t^2;
126 end
127
128 %%% Jacobian fxn
129 function [H] = jcb(x,L,t)
130
131     xi = x(1);
132     xid = x(2);
133     a = x(3);
134     ad = x(4);
135     g = 9.81;
136

```

```

137 H11 = 2*(xid*t+xi-L)/sqrt((2*a-g*t^2+2*ad*t)^2 + 4*(xid*t+xi-L)^2);
138 H12 = H11 * t;
139 H13 = (2*a - g*t^2 + 2*ad*t) / sqrt((2*a-g*t^2+2*ad*t)^2 +
    4*(xid*t+xi-L)^2);
140 H14 = H13*t;
141
142 H21 = 2*(2*a - g*t^2 + 2*ad*t)/((2*a - g*t^2 + 2*ad*t)^2 +
    4*(xid*t+xi-L)^2);
143 H22 = H21*t;
144 H23 = 4*(-xid*t-xi+L)/((2*a - g*t^2 + 2*ad*t)^2 + 4*(xid*t+xi-L)^2);
145 H24 = -t*H23;
146
147 H = [H11 H12 H13 H14;H21 H22 H23 H24];
148 end
149
150 %%% Plotting Functions
151 function [] = plot_NLS(x0t,x0NLS,Y,tvec,R,L)
152
153 N = length(tvec);
154
155 % Get Truth/Estimated Traj
156
157 trajT = zeros(2,N);
158 trajE = zeros(2,N);
159
160
161 for k = 1:N
162     [xi,a] = f(x0t,tvec(k));
163     trajT(:,k) = [xi;a];
164     [xi,a] = f(x0NLS,tvec(k));
165     trajE(:,k) = [xi;a];
166 end
167
168 % Get measurment in x,y coords
169 rho = Y(1:2:end);
170 theta = Y(2:2:end);
171 for i = 1:N
172     aM(i) = rho(i)*sin(theta(i));
173     eM(i) = L - rho(i)*cos(theta(i));
174
175     y_est = h(x0NLS,tvec(i),eye(2),L,false);
176     rho_est(i) = y_est(1);
177     theta_est(i) = y_est(2);
178 end
179
180
181 figure(4); hold on; grid on
182 scatter(trajE(1,:),trajE(2,:), 's', 'MarkerFaceColor', [0.5 0 0.8],
    'MarkerEdgeColor', [0.5 0 0.8])
183 scatter(trajT(1,:),trajT(2,:),100,'x','Color', '#40E0D0')
184 scatter(eM,aM,'ro')
185 legend('NLS Estimate','Ground Truth','Measurments')
186 xlabel("Easting (m)")
187 ylabel("Altitude (m)")

```



```

188     title("NLS Missile Tracking")
189
190     fname = fullfile('Figures',['NLS_estimate' '.png']);
191
192     % Set figure properties for saving
193     fig = gcf;
194     set(fig,'Color','w','PaperPositionMode','auto');
195
196     % Save as PNG (use -r300 for high resolution)
197     print(fig,'-dpng','-r300',fname);
198
199     % Residual Plot
200     figure(6); hold on; grid on
201     tiledlayout(2,1)
202     nexttile;
203     plot(tvec,rho_est(:) - rho,'-o','Color','b')
204     ylabel('\rho residuals')
205     ylim([-2500 1500])
206     nexttile;
207     plot(tvec,theta_est(:) - theta,'-o','Color','r')
208     xlabel("Time (s)")
209     ylabel("\theta residuals")
210
211     fname = fullfile('Figures',['NLS_estimate_Res' '.png']);
212
213     % Set figure properties for saving
214     fig = gcf;
215     set(fig,'Color','w','PaperPositionMode','auto');
216
217     % Save as PNG (use -r300 for high resolution)
218     print(fig,'-dpng','-r300',fname);
219
220     figure(67); hold on; grid on
221     plot(tvec,rho_est(:) - rho,'-o','Color','b')
222     ylabel('\rho residuals')
223     xlabel("Time (s)")
224     ylim([-2500 1500])
225
226     fname = fullfile('Figures',['NLS_Rho_Res' '.png']);
227
228     % Set figure properties for saving
229     fig = gcf;
230     set(fig,'Color','w','PaperPositionMode','auto');
231
232     % Save as PNG (use -r300 for high resolution)
233     print(fig,'-dpng','-r300',fname);
234
235
236 end

```

3.3 Code for Problem 3e

```

1 %%% ASEN 6044 HW2 Q3e

```

```

2
3 %%% Load Datafile
4 clear; clc; close all
5 load("hw2_missileprob_data.mat")
6
7 %%% Constants
8 N = size(ystacked,1)/2;
9 np = 2;
10 L = 80000;
11 dt = 5; % sec
12 t = dt:dt:N*dt;
13
14 fun = @(x) Likelihood(x, ystacked,t);
15 options =
16     optimoptions('fminunc','Algorithm','trust-region','SpecifyObjectiveGradient',true,'
17     30000, 'MaxIterations', 10000);
18
19 x0 = [50;300;20;100];
20
21 [x,fval] = fminunc(fun,x0,options);
22
23 %%% Plot Results
24 err = x0true - x %% ERROR
25 results(x0true,x,ystack,t,L)
26
27 function [Like,G] = Likelihood(x,Y,t)
28
29 % Unpack
30 xi = x(1);
31 xid = x(2);
32 a = x(3);
33 ad = x(4);
34 rho = Y(1:2:end);
35 theta = Y(2:2:end);
36
37 % Constants
38 nu = 1/2;
39 sigma2 = 0.005;
40 L = 80000;
41 g = 9.81;
42
43 % Dynamics
44 xit = xi + xid*t;
45 at = a+ad*t-0.5*g*t.^2;
46
47 % Force col vector
48 xit = xit(:);
49 at = at(:);
50
51 % P(rho_k|x0)
52
53 % Constant terms
54 C = gamma(((nu+1)/2))/gamma(nu/2) * (1/(sqrt(nu*pi)));

```

```

54
55 Pr = C*((1 + ((rho - sqrt((L - xit).^2+(at).^2)).^2)/nu)).^(-(nu+1)/2);
56
57 % P(theta_k|x0)
58
59 % Constant term
60 C = 1/(sqrt(sigma2*2*pi));
61
62 Pb = C*exp((- (theta-atan2(at,(L-xit))).^2)/(2*sigma2));
63
64 Like = -sum(log(Pr.*Pb));
65
66 %% Gradient Vector
67 if nargout > 1
68     t = t(:);
69     % First Vector
70     C1 = (nu+1)/2;
71
72     G11 = (2*(rho - sqrt((L - xit).^2+(at).^2)).*(L-xit))./((((rho -
        sqrt((L - xit).^2+(at).^2)).^2) + nu).*(sqrt((L -
        xit).^2+(at).^2)));
73     G12 = G11.*t;
74     G13 = (-2*(rho - sqrt((L - xit).^2+(at).^2)).*at)./((((rho - sqrt((L
        - xit).^2+(at).^2)).^2) + nu).*(sqrt((L - xit).^2+(at).^2)));
75     G14 = G13.*t;
76
77     G11 = sum(G11);
78     G12 = sum(G12);
79     G13 = sum(G13);
80     G14 = sum(G14);
81
82     C2 = 1/(2*sigma2);
83     G21 = (-2*(theta-atan2(at,(L-xit))).*at)./(((at./(L-xit)).^2 +
        1).*(L-xit).^2);
84     G22 = G21.*t;
85     G23 = (-2*(theta-atan2(at,(L-xit))).)./(((at./(L-xit)).^2 +
        1).*(L-xit));
86     G24 = G23.*t;
87
88     G21 = sum(G21);
89     G22 = sum(G22);
90     G23 = sum(G23);
91     G24 = sum(G24);
92
93     G = C1*[G11;G12;G13;G14]+ C2*[G21;G22;G23;G24];
94 end
95 end
96
97 %% Plotting Functions
98 function [] = results(x0t,x0_hat,Y,tvec,L)
99
100     N = length(tvec);
101
102     % Get Truth/Estimated Traj

```

```

103
104     trajT = zeros(2,N);
105     trajE = zeros(2,N);
106
107
108     for k = 1:N
109         [xi,a] = f(x0t,tvec(k));
110         trajT(:,k) = [xi;a];
111         [xi,a] = f(x0_hat,tvec(k));
112         trajE(:,k) = [xi;a];
113     end
114
115     rho = Y(1:2:end);
116     theta = Y(2:2:end);
117     % Get measurment in x,y coords
118     for i = 1:N
119         aM(i) = rho(i)*sin(theta(i));
120         eM(i) = L - rho(i)*cos(theta(i));
121
122         y_est = h(x0_hat,tvec(i),L);
123         rho_est(i) = y_est(1);
124         theta_est(i) = y_est(2);
125     end
126
127
128     figure(2); hold on; grid on
129     scatter(trajE(1,:),trajE(2,:), 's', 'MarkerFaceColor', [0.5 0 0.8],
130             'MarkerEdgeColor', [0.5 0 0.8])
131     scatter(trajT(1,:),trajT(2,:),100,'x','Color', '#40E0D0')
132     scatter(eM,aM,'ro')
133     legend('ML Estimate','Ground Truth','Measurments')
134     xlabel("Easting (m)")
135     ylabel("Altitude (m)")
136     title("ML Missile Tracking")
137
138     fname = fullfile('Figures',['ML_estimate' '.png']);
139
140     % Set figure properties for saving
141     fig = gcf;
142     set(fig,'Color','w','PaperPositionMode','auto');
143
144     % Save as PNG (use -r300 for high resolution)
145     print(fig,'-dpng','-r300',fname);
146
147     % Residual Plot
148     figure(3); hold on; grid on
149     tiledlayout(2,1)
150     nexttile;
151     plot(tvec,rho_est(:) - rho,'-o','Color','b')
152     ylabel('\rho residuals')
153     ylim([-300 100])
154
155     nexttile;
156     plot(tvec,theta_est(:) - theta,'-o','Color','r')

```

```

156     xlabel("Time (s)")
157     ylabel("\theta residuals")
158
159     fname = fullfile('Figures', ['ML_estimate_Res' '.png']);
160
161     % Set figure properties for saving
162     fig = gcf;
163     set(fig, 'Color', 'w', 'PaperPositionMode', 'auto');
164
165     % Save as PNG (use -r300 for high resolution)
166     print(fig, '-dpng', '-r300', fname);
167
168     figure(67); hold on; grid on
169     plot(tvec, rho_est(:) - rho, '-o', 'Color', 'b')
170     ylabel('\rho residuals')
171     xlabel("Time (s)")
172     ylim([-300 100])
173
174     fname = fullfile('Figures', ['ML_Rho_Res' '.png']);
175
176     % Set figure properties for saving
177     fig = gcf;
178     set(fig, 'Color', 'w', 'PaperPositionMode', 'auto');
179
180     % Save as PNG (use -r300 for high resolution)
181     print(fig, '-dpng', '-r300', fname);
182
183
184 end
185
186 function [y] = h(x0,t, L)
187
188
189     vk = [0 0];
190
191     [xik,ak] = f(x0,t);
192
193     rho = sqrt((L - xik)^2 + ak^2) + vk(1);
194     theta = atan2(ak, (L-xik)) + vk(2);
195
196     y = [rho;theta];
197 end
198
199 %%% Dynamics function
200 function [xik,ak] = f(x0,t)
201
202     xi0 = x0(1);
203     xid0 = x0(2);
204     a0 = x0(3);
205     ad0 = x0(4);
206
207     g = 9.81;
208
209     xik = xi0 + xid0*t;

```

```
210     ak = a0 + ad0*t - 0.5*g*t^2;  
211 end
```